

UNIVERSITAS DIAN NUSWANTORO (UDINUS)

FAKULTAS ILMU KOMPUTER

PROGRAM STUDI S1 TEKNIK INFORMATIKA • REKAYASA PERANGKAT LUNAK

TUGAS KELOMPOK AKADEMIK

## ANALISIS MODEL PROSES AGILE DAN PENGEMBANGAN SISTEM KOSTKU

Kajian Teori Ekstensif Agile Software Models (XP, Scrum, DSDM) Serta Penentuan & Justifikasi Model Proses Rekayasa Perangkat Lunak Platform KostKu

Mata Kuliah: Rekayasa Perangkat Lunak (RPL)

Disusun Oleh (Kelompok) : 1. Oscar Herdian Wijaya (A11.2025.16309)  
2. Maulana Hadi Saputra (A11.2025.16307)  
3. Angelo Joe Lara Anugrah Wisanggeni Putra Tudjiyo (A11.2025.16337)

Program Studi / Kelas : S1 Teknik Informatika / A11

Dosen Pengampu : Dosen Pengampu Mata Kuliah Rekayasa Perangkat Lunak

Tanggal Pengumpulan : Oktober 2026

SEMARANG • JAWA TENGAH • 2026

## BAGIAN 1: IDE PENGEMBANGAN PERANGKAT LUNAK (KOSTKU)

### 1.1 Identifikasi & Deskripsi Ide Perangkat Lunak

Perangkat lunak yang dikembangkan diberi nama **KostKu** (*Smart Boarding Ecosystem & Architectural Blueprint Engine*). **KostKu** dirancang sebagai ekosistem digital dua sisi (*two-sided platform*) terpadu yang memadukan **Marketplace Pencarian Kost Interaktif** untuk pencari kos (mahasiswa dan perantau) serta **Back-Office Property Management System** untuk pemilik atau pengelola indekos.

Latar belakang perancangan KostKu berangkat dari disparitas ekspektasi nyata (*reality gap*) dan friksi operasional di industri persewaan indekos Indonesia:

- **Friksi Pencari Kos:** Maraknya manipulasi visual berupa foto kamar berlensa sudut lebar (*wide-angle*) yang mendistorsi persepsi ruang nyata, ketidakpastian apakah perabot bawaan (misal kasur 160×200 cm atau meja kerja) akan muat, serta minimnya transparansi ventilasi udara alami. Hal ini memaksa pencari kos menghabiskan waktu dan biaya untuk survei fisik yang kerap mengecewakan.
- **Friksi Pengelola Kos:** Pembukuan manual berbasis buku tulis yang rentan rusak/hilang, sengketa penagihan utilitas listrik bulanan akibat keterlambatan mencatat angka kWh meteran, komplain fasilitas yang tercecer di chat WhatsApp pribadi, serta potongan komisi perantara aplikasi konvensional yang mencekik (5% hingga 15% per transaksi).

### 1.2 Kemampuan & Fitur Unggulan Perangkat Lunak (Feature Capabilities)

KostKu mengintegrasikan inovasi teknologi rekayasa web modern untuk menghadirkan keunggulan kompetitif yang memecahkan masalah di atas:

#### 1. Architectural 2D Blueprint Engine

&lt;15 KB SVG

Mesin visualisasi denah arsitektur interaktif berbasis vektor SVG presisi skala **1:50**. Menggambarkan dimensi riil dinding, bukaan pintu, posisi jendela, meja kerja, dan kasur springbed secara akurat tanpa distorsi kamera. Berukuran ultra-ringan (<15 KB) sehingga termuat seketika bahkan pada koneksi seluler hemat daya.

#### 2. 3D Isometric Cutaway Viewer

THREE.JS / WEBGL

Visualisasi kamar 3D isometrik dengan dinding terpotong (*cutaway*) yang memungkinkan calon penyewa memutar perspektif dari **4 sudut pandang** (Isometrik Kanan, Kiri, Top-Down 45°, Depan) serta beralih antara **Mode Siang** (*natural skylight*) dan **Mode Malam** (*warm 2700K ambient glow*).

#### 3. AI Room Scanner & Questionnaire

SPATIAL AI

Pemilik kos cukup mengunggah foto kamar tidur. AI Vision menganalisis batas ruang dan menyajikan kuesioner interaktif (ukuran kasur Single 90×200 s.d King 180×200, lemari, meja, kamar mandi dalam), kemudian men-*generate* denah arsitektur 2D dan model 3D secara otomatis tanpa keahlian CAD.

#### 4. Anti-Bot 1 KTP 1 Akun & Validasi NIK

ANTI-FRAUD

Menjamin integritas dan keamanan ekosistem sewa melalui validasi NIK 16 digit resmi kependudukan via endpoint `/api/check-nik`, foto fisik e-KTP, verifikasi koordinat GPS Google Maps, dan fitur pemulihan akun mandiri (*account recovery*) berbasis NIK untuk mencegah akun bot/penipuan.

#### ⚡ Otomatisasi Utilitas & 1-Click WhatsApp Dispatcher

Kalkulator matematis terintegrasi: **Biaya Listrik =  $\text{Max}(0, \text{Meter Akhir} - \text{Meter Awal}) \times \text{Tarif per kWh}$** . Menghasilkan rincian sewa transparan yang langsung dikirimkan ke WhatsApp penyewa melalui *deep-link formatting* (`wa.me`) dalam 1-klik tanpa pengetikan manual.

#### ⚙ Standar Operasional Prosedur (SOP) Enterprise Terintegrasi

Dilengkapi 3 protokol operasional baku: (1) **SOP Maintenance Terjadwal** pada jam sepi (01.00–04.00) didahului banner 24 jam & full backup data; (2) **SOP Server Down & Error Kritis** dengan auto-monitoring <2m, failover halaman darurat <5m, dan rollback <15m; serta (3) **SOP Komplain Pengguna** dengan klasifikasi Mendesak (SLA respon 15–30 menit, selesai 2–4 jam) dan Biasa (SLA 1×24 jam).

### 1.3 Penentuan Model Software Proses yang Sesuai: Scrum Framework

Berdasarkan karakteristik teknis sistem, dinamika kebutuhan pengguna, serta batasan sumber daya tim pengembang mahasiswa, model proses perangkat lunak yang dipilih dan diterapkan untuk pengembangan **KostKu** adalah **Agile Software Development Model dengan Framework SCRUM**, yang diperkaya dengan praktik teknis terpilih dari **Extreme Programming (XP)**.

#### A. Alasan Pemilihan Framework Scrum untuk KostKu

1. **Kebutuhan Iterasi Pendek Berbasis Umpan Balik Cepat (Rapid Feedback Loop):** KostKu memiliki dua persona pengguna yang sangat berbeda (mahasiswa melek teknologi vs bapak/ibu kos lansia). Perubahan antarmuka dan penyesuaian alur kerja memerlukan validasi berkala. Siklus *Sprint* berdurasi **2 minggu** memungkinkan tim merilis fungsionalitas bertahap (Sprint 1 Denah 2D, Sprint 2 Kalkulator Utilitas, Sprint 3 Verifikasi KTP, Sprint 4 AI Scanner) dan langsung menguji kegunaannya di lapangan.
2. **Tingginya Ketidakpastian Kebutuhan & Eksplorasi Inovasi Baru:** Integrasi AI Vision Room Scanner dan rendering 3D WebGL di browser merupakan domain eksploratif. Menggunakan model sekuensial linier (seperti Waterfall) sangat berisiko gagal karena spesifikasi teknis AI dan grafis belum dapat dikunci di awal. Scrum memfasilitasi *Sprint Backlog Grooming* untuk merevisi prioritas sesuai kemajuan teknis.
3. **Struktur Tim Pengembang Skala Kecil yang Lintas-Fungsi (Cross-Functional Team):** Tim proyek terdiri dari mahasiswa yang memegang peran jamak (Frontend, Backend, UI/UX, dan QA). Scrum menyediakan kerangka kerja akuntabilitas harian (*Daily Scrum*) yang menjaga sinkronisasi tugas tanpa birokrasi dokumentasi tebal.
4. **Penggabungan Praktik Rekayasa XP (Extreme Programming Hybrids):** Tim mengadopsi teknik *Continuous Integration* (CI) via GitHub Actions, *Pair Programming* pada modul kritis (algoritma selisih kWh dan parsing SVG), serta *Refactoring* berkelanjutan guna memastikan kode tetap modular, bersih, dan minim regresi.

#### B. Keuntungan Penggunaan Scrum pada Proyek KostKu

- **Time-to-Market Sangat Cepat:** Produk dapat dirilis ke lingkungan pengujian pengguna (*MVP - Minimum Viable Product*) hanya dalam 1 siklus *Sprint* pertama (versi fungsional marketplace publik).
- **Fleksibilitas Tinggi terhadap Perubahan Kebutuhan (Agility):** Ketika muncul masukan penting (seperti kebutuhan fitur SOP Server Down atau pengetatan validasi NIK anti-bot), kebutuhan tersebut dapat segera dimasukkan ke dalam *Product Backlog* untuk dijadwalkan pada *Sprint* berikutnya tanpa mengacaukan arsitektur inti.
- **Transparansi dan Visibilitas Progres Terukur:** Penggunaan papan kanban (*Scrum Board*), estimasi *Story Points*, dan grafik *Sprint Burndown* memberikan kepastian matriks ketercapaian target kepada seluruh anggota tim dan dosen pembimbing.
- **Peningkatan Kualitas Perangkat Lunak Berkelanjutan:** Setiap akhir *sprint* diwajibkan melewati sesi *Sprint Review* dan *Sprint Retrospective* untuk mengevaluasi hambatan komunikasi teknis dan memperbaiki efisiensi kerja tim.

#### C. Kerugian & Mitigasi Risiko Penggunaan Scrum pada Proyek KostKu

- **Risiko Scope Creep (Perluasan Fitur Tanpa Kendali):** Fleksibilitas Scrum dapat memicu penambahan ide fitur baru secara terus-menerus sehingga peluncuran final tertunda.  
*Mitigasi:* Product Owner menerapkan teknik prioritas MoSCoW dan menegakkan *Definition of Done (DoD)* secara ketat sebelum fitur baru diizinkan masuk ke *sprint*.
- **Ketergantungan Kuat pada Komitmen & Kolaborasi Anggota Tim:** Jika salah satu anggota pasif atau absen pada *Daily Scrum*, koordinasi modul frontend-backend dapat tersendat.  
*Mitigasi:* Membagi antarmuka komunikasi API secara terstandarisasi sejak awal (REST contract) dan menerapkan repositori terpusat.
- **Dokumentasi Arsitektur Berpotensi Terabaikan:** Fokus berlebih pada kode yang berjalan cepat (*working software*) kerap membuat diagram UML dan catatan pemeliharaan terlewat.  
*Mitigasi:* Memasukkan penyusunan dokumentasi teknis dan blueprint sistem sebagai salah satu klausul wajib dalam *Definition of Done (DoD)*.

## BAGIAN 2: TEORI DAN SUMBER PENDUKUNG MODEL AGILE

### 2.1 Pengantar & Paradigma Agile Software Development

Paradigma **Agile** lahir sebagai respon kritis terhadap kegagalan model sekuensial tradisional (*plan-driven models* seperti Waterfall) yang kaku, membutuhkan dokumentasi masif di awal, dan lambat merespon perubahan pasar. Dirumuskan dalam *Agile Manifesto* (Beck et al., 2001), pendekatan Agile menempatkan adaptabilitas, kolaborasi aktif, rilis perangkat lunak fungsional bertahap, dan kepuasan pengguna di atas rencana kaku yang telah ditentukan sebelumnya.

### 2.2 Extreme Programming (XP)

**Extreme Programming (XP)** adalah metodologi pengembangan perangkat lunak berbasis Agile yang dicetuskan oleh **Kent Beck** pada akhir 1990-an saat memimpin proyek *Chrysler Comprehensive Compensation System (C3)*. XP berfokus ekstrem pada keunggulan teknis rekayasa perangkat lunak (*software engineering excellence*) dan efisiensi penulisan kode untuk menghadapi perubahan kebutuhan yang sangat dinamis.

#### A. Lima Nilai Inti Extreme Programming (XP Core Values)

- 1. Communication:** Menghilangkan sekat komunikasi formal antar-pengembang dan pemangku kepentingan melalui interaksi lisan langsung setiap hari.
- 2. Simplicity:** Menganut prinsip "Lakukan hal paling sederhana yang mungkin bekerja hari ini" (*Do the simplest thing that could possibly work*). Menghindari arsitektur spekulatif (YAGNI).
- 3. Feedback:** Mengoptimalkan perolehan umpan balik instan melalui pengujian otomatis skala menit, integrasi harian, dan demonstrasi berkala.
- 4. Courage:** Berani melakukan refactoring kode berskala besar, membuang kode usang yang tidak efisien, dan memperbaiki kekeliruan desain sejak dini.
- 5. Respect:** Saling menghormati kontribusi rekan kerja, menjamin kualitas kerja tim, dan tidak merusak integritas kode bersama.

#### B. Dua Belas Praktik Rekayasa Inti XP (12 Core Engineering Practices)

| Kategori           | Praktik XP             | Deskripsi Operasional                                                                                                  |
|--------------------|------------------------|------------------------------------------------------------------------------------------------------------------------|
| Coding & Design    | Test-Driven Dev (TDD)  | Menulis tes unit otomatis terlebih dahulu sebelum kode implementasi ditulis, memastikan code coverage tinggi.          |
|                    | Refactoring            | Penyempurnaan struktur internal kode secara berkala tanpa mengubah perilaku eksternal demi menjaga keterbacaan kode.   |
|                    | Simple Design          | Desain kode dibuat sesederhana mungkin untuk memenuhi kebutuhan saat ini tanpa kompleksitas arsitektur spekulatif.     |
|                    | System Metaphor        | Menggunakan analogi bersama yang disepakati tim untuk menggambarkan alur kerja sistem secara mudah dipahami.           |
| Teamwork           | Pair Programming       | Dua programmer bekerja bersama pada satu workstation: satu pembuat kode (Driver) dan satu peninjau logika (Navigator). |
|                    | Collective Ownership   | Seluruh anggota tim memiliki hak dan kewajiban untuk memperbaiki bagian mana pun dari kode tanpa birokrasi izin.       |
|                    | Coding Standards       | Menetapkan konvensi penulisan sintaks seragam agar kode terasa seperti ditulis oleh satu orang.                        |
|                    | Sustainable Pace       | Menjaga batas jam kerja wajar (40-Hour Week) untuk mencegah kelelahan mental yang memicu kecacatan bug.                |
| Feedback & Release | Continuous Integration | Penggabungan kode ke repositori utama dilakukan beberapa kali sehari disertai automated unit testing instan.           |
|                    | Small Releases         | Merilis pembaruan perangkat lunak fungsional dalam siklus sangat singkat (1–3 minggu) ke tangan pengguna.              |
|                    | On-site Customer       | Perwakilan klien berada langsung di ruang kerja tim untuk menjawab pertanyaan spesifikasi dan menguji hasil rilis.     |
|                    | Planning Game          | Pertemuan perencanaan di mana klien menentukan User Stories dan pengembang memperkirakan estimasi teknis.              |

## 2.3 Scrum Framework

Diciptakan oleh **Ken Schwaber** dan **Jeff Sutherland** pada awal 1990-an dan dibukukan resmi dalam *The Scrum Guide*, **Scrum** adalah kerangka kerja manajemen adaptif yang ringan untuk menghasilkan solusi bernilai tinggi secara berulang bagi masalah-masalah yang kompleks. Scrum berakar kuat pada teori kontrol proses empiris (**Empiricism**) dan pemikiran ramping (**Lean Thinking**).

### A. Tiga Pilar Empirisme Scrum (Three Pillars of Empiricism)

1. **Transparansi (Transparency):** Proses dan hasil kerja yang sedang berlangsung harus terlihat jelas dan nyata baik bagi pelaksana kerja maupun penerima hasil kerja. Tidak ada informasi tersembunyi.
2. **Inspeksi (Inspection):** Artefak dan kemajuan proyek Scrum harus diinspeksi secara berkala dan cermat untuk mendeteksi deviasi yang tidak diinginkan sedini mungkin.
3. **Adaptasi (Adaptation):** Jika ditemukan aspek proses yang melenceng di luar batas yang dapat diterima, penyesuaian proses atau materi yang dihasilkan harus segera dilakukan tanpa menunda.

### B. Tiga Peran Akuntabilitas dalam Scrum (Scrum Roles)

- **Product Owner (PO):** Bertanggung jawab memaksimalkan nilai bisnis produk yang dihasilkan. PO memiliki akuntabilitas tunggal atas pengelolaan efektif *Product Backlog* (menyusun, mengurutkan, dan memperjelas butir kebutuhan).
- **Scrum Master (SM):** Pemimpin yang melayani (*servant leader*) yang bertindak sebagai fasilitator dan pelatih efektivitas tim Scrum. SM bertugas menghilangkan hambatan kerja (*impediments*) dan memastikan nilai-nilai Scrum dipahami serta dijalankan.
- **Developers (Tim Pengembang):** Sekelompok profesional lintas-keahlian (desainer, programmer, analis, tester) yang berkomitmen untuk menciptakan aspek *Increment* yang dapat digunakan pada setiap akhir Sprint.

### C. Lima Acara / Seremoni Scrum (Scrum Events)

#### 1. The Sprint

WADAH UTAMA (1–4 MINGGU)

Jantung dari Scrum berupa wadah berdurasi tetap (*time-box*) konsisten di mana ide diubah menjadi nilai nyata. Sprint baru segera dimulai tepat setelah Sprint sebelumnya selesai.

#### 2. Sprint Planning

MAKS. 8 JAM (SPRINT 1 BULAN)

Acara pembuka Sprint untuk menetapkan tujuan (*Sprint Goal*), memilih butir backlog dari Product Backlog, dan merencanakan bagaimana pekerjaan teknis akan diselesaikan.

#### 3. Daily Scrum

15 MENIT SETIAP HARI

Pertemuan harian 15 menit bagi Developers untuk menginspeksi kemajuan menuju *Sprint Goal* dan menyesuaikan rencana kerja untuk 24 jam ke depan guna menghilangkan hambatan.

#### 4. Sprint Review & Retrospective

EVALUASI AKHIR  
SPRINT

**Review:** Tim mendemonstrasikan hasil kerja kepada pemangku kepentingan. **Retrospective:** Tim mengevaluasi proses kerja, relasi anggota, dan merencanakan efisiensi.

### D. Tiga Artefak Scrum & Komitmen Kualitas

- **Product Backlog:** Daftar terurut kebutuhan, perbaikan, dan fitur yang dibutuhkan oleh produk (Komitmen: *Product Goal*).
- **Sprint Backlog:** Sekumpulan butir Product Backlog terpilih untuk dikerjakan dalam satu Sprint, ditambah rencana kerja operasional (Komitmen: *Sprint Goal*).
- **Increment:** Langkah konkret menuju tercapainya Product Goal. Sebuah increment bernilai guna tinggi dan memenuhi kriteria kualitas formal tim (Komitmen: *Definition of Done / DoD*).

2.4 Dynamic Systems Development Model (DSDM)

DSDM didirikan pada tahun 1994 di Inggris oleh konsorsium vendor dan pengguna industri perangkat lunak (sekarang dikenal sebagai *Agile Business Consortium*). DSDM dirancang untuk menyediakan tata kelola terstruktur bagi pengembangan aplikasi cepat (*Rapid Application Development / RAD*) tanpa mengorbankan kualitas dan disiplin proyek tingkat enterprise.

A. Filosofi Inti & Pembalikan Segitiga Proyek Tradisional

Filosofi fundamental DSDM menyatakan: *"Setiap proyek harus selaras dengan tujuan bisnis strategis dan memberikan manfaat nyata secara cepat."* Dalam manajemen proyek tradisional (Waterfall), **Fungsionalitas / Fitur (Scope)** dikunci di awal sebagai target tetap, sedangkan **Waktu (Time)** dan **Biaya (Cost)** bersifat variabel yang kerap membengkak. Sebaliknya, DSDM membalik paradigma tersebut: **Waktu, Biaya, dan Kualitas bersifat TETAP (Fixed)**, sedangkan **Fungsionalitas / Fitur bersifat VARIABEL (Contingency)** yang dikelola melalui teknik prioritas MoSCoW.

B. Delapan Prinsip Utama DSDM (8 Core Principles)

- Focus on the business need:** Setiap keputusan pengembangan harus memberikan nilai bisnis riil yang dapat diukur.
- Deliver on time:** Menjaga batas waktu rilis secara ketat melalui penetapan *timebox* yang tidak dapat ditawar.
- Collaborate:** Melibatkan pengguna bisnis dan pengembang secara terpadu sepanjang siklus proyek.
- Never compromise quality:** Kualitas sistem telah disepakati di awal dan tidak boleh diturunkan demi mengejar target waktu.
- Build incrementally from firm foundations:** Membangun fondasi arsitektur dan kelayakan yang solid sebelum iterasi fitur dimulai.
- Develop iteratively:** Menerapkan siklus perbaikan terus-menerus berdasarkan umpan balik pengguna langsung.
- Communicate continuously and clearly:** Mengutamakan komunikasi visual, modeling, dan demonstrasi dibanding dokumen tebal.
- Demonstrate control:** Pengelolaan proyek proaktif yang terukur menggunakan metrik transparansi berkala.

C. Metode Prioritisasi MoSCoW dalam DSDM

DSDM menerapkan aturan alokasi sumber daya berbasis teknik MoSCoW untuk menjamin pengiriman tepat waktu:

| Prioritas       | Definisi Kebutuhan                                                                                                 | Implikasi Terhadap Rilis                                               | Alokasi Beban                     |
|-----------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|-----------------------------------|
| Must Have (M)   | Fitur fundamental yang wajib ada. Tanpa fitur ini, sistem tidak bernilai dan tidak layak diluncurkan.              | Ketidakberadaan fitur membatalkan kelayakan rilis.                     | Maksimal 60% kapasitas tim.       |
| Should Have (S) | Fitur penting yang sangat diharapkan keberadaannya, namun sistem masih bisa beroperasi jika ada solusi alternatif. | Fitur penting nomor dua, dikerjakan setelah seluruh Must Have selesai. | Sekitar 40% (Penyangga / buffer). |
| Could Have (C)  | Fitur pelengkap yang diinginkan jika waktu dan sumber daya masih tersisa. Berdampak bisnis lebih rendah.           | Fitur pertama yang dipangkas jika tenggat waktu terancam meleset.      |                                   |
| Won't Have (W)  | Fitur yang disepakati untuk tidak dikerjakan pada timebox saat ini, dialokasikan untuk fase mendatang.             | Mencegah scope creep dan menjaga fokus rilis tim saat ini.             | 0% (Rilis mendatang).             |

D. Empat Fase Siklus Hidup DSDM (DSDM Lifecycle)

- Pre-Project:** Inisiasi proyek untuk mengidentifikasi sponsor dan lingkup ide bisnis secara ringkas.
- Feasibility & Foundations:** Investigasi kelayakan teknis/bisnis serta perumusan fondasi arsitektur dan tata kelola proyek.
- Evolutionary Development:** Iterasi gabungan antara eksplorasi desain, pembuatan kode fungsional, dan pengujian terintegrasi.
- Deployment & Post-Project:** Penerapan sistem ke lingkungan produksi pengguna dan evaluasi manfaat bisnis aktual pasca-rilis.

## 2.5 Matriks Komparasi Komprehensif: XP vs Scrum vs DSDM

| Parameter                    | Extreme Programming (XP)                                                           | Scrum Framework                                                                  | DSDM (Dynamic Systems)                                                                   |
|------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Fokus Utama</b>           | Praktik keunggulan teknis rekayasa kode (TDD, pair programming, refactoring).      | Kerangka kerja manajemen adaptif, transparansi empiris, dan produktivitas tim.   | Tata kelola proyek enterprise terstruktur dan keselarasan nilai bisnis strategis.        |
| <b>Durasi Iterasi</b>        | Sangat pendek (1 hingga 2 minggu).                                                 | Pendek dan konsisten (1 hingga 4 minggu; umumnya 2 minggu).                      | Terbagi dalam <i>Timebox</i> fleksibel (2 minggu hingga 6 minggu per timebox).           |
| <b>Aturan Praktik Teknis</b> | Sangat ketat dan preskriptif (Wajib TDD, Pair Programming, CI, Simple Design).     | Agnostik / Terbuka (Tidak mewajibkan teknik coding tertentu, diserahkan ke tim). | Menyediakan panduan terstruktur (Modeling, prototyping, configuration management).       |
| <b>Keterlibatan Klien</b>    | <i>On-site Customer</i> (Pengguna berada langsung bersama tim setiap hari).        | Diwakili secara penuh oleh peran <i>Product Owner</i> dalam tim Scrum.           | Keterlibatan formal dari peran bisnis ( <i>Business Ambassador</i> , <i>Visionary</i> ). |
| <b>Manajemen Kebutuhan</b>   | User Stories, Planning Game, refactoring instan saat ada cerita baru.              | Product Backlog, Sprint Planning, perubahan ditunda ke Sprint berikutnya.        | Metode prioritasasi MoSCoW (Must, Should, Could, Won't have).                            |
| <b>Kelebihan Utama</b>       | Kualitas basis kode sangat tinggi, bug sangat minim karena TDD & pair programming. | Sangat adaptif, mudah diadopsi tim multidisiplin, visibilitas progres tinggi.    | Kepastian tenggat waktu dan anggaran sangat kuat, cocok untuk korporasi besar.           |
| <b>Keterbatasan</b>          | Memerlukan kedisiplinan teknis tinggi, rentan kelelahan mental jika dipaksakan.    | Tidak memberikan panduan rekayasa teknis; rentan degradasi kode jika tim pemula. | Beban tata kelola dan peran formal lebih banyak dibanding Scrum atau XP.                 |

## 2.6 Kesimpulan Analisis & Rekomendasi Terapan untuk KostKu

Dari perbandingan ketiga model, sintesis terbaik untuk proyek **KostKu** adalah menerapkan **Scrum sebagai kerangka kerja tata kelola proyek utama** (memanfaatkan siklus Sprint 2 minggu, upacara Daily Standup, dan Sprint Review untuk mengakomodasi kebutuhan pengguna kos), kemudian memperkuatnya dengan **praktik teknis terpilih dari XP** (seperti Test-Driven Development pada modul kalkulator utilitas dan Continuous Integration pada pipeline build multi-platform). Pendekatan hibrida ini memberikan fleksibilitas manajerial sekaligus kepastian mutu kode tingkat tinggi.

## DAFTAR PUSTAKA & SUMBER REFERENSI PENDUKUNG

- [1] Beck, K., et al. (2001). *Manifesto for Agile Software Development*. Agile Alliance.
- [2] Beck, K., & Andres, C. (2004). *Extreme Programming Explained: Embrace Change* (2nd Edition). Addison-Wesley Professional.
- [3] Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum*. Scrum.org.
- [4] Agile Business Consortium. (2014). *The DSDM Agile Project Framework*. Agile Business Consortium Limited.
- [5] Sommerville, I. (2016). *Software Engineering* (10th Edition). Pearson Education. (Chapter 3: Agile Software Engineering).
- [6] Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th Edition). McGraw-Hill.
- [7] Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd Edition). Addison-Wesley.
- [8] Rubin, K. S. (2012). *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Addison-Wesley.